QuadNet

S.I.G.S.M.

Arquitectura y funcionamiento del código fuente

Grupo QuadNet

Rol	Apellido	Nombre	C.I	Email
Coordinador	Raffetto	Guillermo	57149245	guillermoraffetto@gmail.com
Sub-Coordinador	Rossello	Matias	57509235	mrossello@estudiante.ceibal.edu.uy
Integrante 1	Blengini	Thiago	57543982	thiagoblengini@gmail.com
Integrante 2	Maciel	Luciano	57075826	lucianosantiago.maciel@estudiante.ceibal.edu.uy

Docente: Romero, Carlos

Fecha de culminación

09/11/2026

TERCERA ENTREGA

I.S.B.O. **3MN**

ÍNDICE

ÍNDICE **1**
1. Introducción **2**
1.1 Esquema conceptual 2
1.2 Principios de organización 3
2. Árbol completo del proyecto **4**
3. Backend: configuración, datos y reglas de negocio **7**
3.1 Backend/config/ 7
3.2 Backend/models/ 7
3.3 Backend/database/bd.sql 11
4. API: puntos de entrada HTTP **12**
4.1 API/usuarios/ 12
4.2 API/funcionarios/ 12
4.3 API/vehiculos/ 13
4.4 API/traslados/ 13
4.5 API/categorias/ y API/subcategorias/ 13
4.6 API/documentos/ 14
4.7 API/encuestas/ 14
4.8 API/portal/ 15
4.9 API/seguridad/ 15
5. Frontend: interfaz y lógica en el navegador **16**
5.1 Páginas de entrada 16
5.2 Frontend/admin/ 16
5.3 Frontend/traslados/ 17
5.4 Frontend/documentos/ 17
5.5 Frontend/dashboard/ 17
5.6 Frontend/js/ 18
5.7 Frontend/css/ y Frontend/img/ 18
6. Infraestructura, ejecución y almacenamiento **19**

1. Introducción

SIGSM es una aplicación web destinada a apoyar dos procesos del Hospital de Clínicas: la gestión y trazabilidad de traslados y la administración y publicación de documentación para pacientes. Incluye, además, administración de usuarios, funcionarios y vehículos, un portal público de documentos y encuestas de satisfacción. El proyecto está construido con HTML, CSS y JavaScript en el navegador, PHP en el servidor y MySQL como base de datos. Su estructura distingue la interfaz (Frontend), los puntos de acceso HTTP (API), los modelos de datos y la configuración (Backend), además de los archivos de infraestructura y almacenamiento. Este documento describe la implementación presente en el ZIP, no una arquitectura idealizada. El sistema sigue una organización por capas inspirada en MVC, pero no es un MVC estricto: una parte considerable del JavaScript se encuentra dentro de las páginas HTML y existen endpoints PHP que realizan consultas SQL directamente.

1.1 Esquema conceptual

```
USUARIO (navegador)
     |
     v
Frontend/       HTML + CSS + JavaScript
                fetch(), formularios y tablas
     |
     v
API/            Endpoints PHP, validación, sesión y respuestas
     |
     v
Backend/models/   Reglas de negocio + consultas SQL
     |
     v
Backend/config/database.php   PDO (Singleton)
     |
     v
MySQL           Datos relacionados y persistentes
```

Los servicios de autenticación, protección de documentos y configuración de Apache se aplican transversalmente. Los archivos PDF se almacenan en disco y se sirven mediante endpoints PHP, en lugar de publicarse como recursos estáticos de acceso directo.

1.2 Principios de organización

Carpeta	Responsabilidad predominante	Qué evita
Frontend/	Presentar y recoger información; iniciar solicitudes HTTP	Exponer credenciales o ejecutar SQL desde el navegador
API/	Interpretar peticiones, validar entradas, controlar acceso y responder	Acoplar cada pantalla a la implementación interna de los modelos
Backend/models	Operar sobre entidades, aplicar reglas y consultar/modificar MySQL	Duplicar lógica de datos en múltiples endpoints
Backend/config	Proporcionar conexión, autenticación y protección de archivos	Repetir configuraciones de seguridad y conexión
Backend/database/	Definir tablas, restricciones y datos iniciales	Depender de la creación manual del esquema
docker/ y archivos raíz	Ejecutar la aplicación y sus dependencias	Configuraciones de despliegue dispersas
uploads/	Conservar los PDF recibidos	Guardar archivos binarios dentro de las filas de las tablas

2. Árbol completo del proyecto

El paquete contiene 120 archivos (sin contar entradas de directorio). El árbol siguiente muestra el código y los archivos de configuración; uploads/documentos/ se crea o utiliza durante la ejecución y no está incluido como archivo en el ZIP.

```
SIGSM/
├── API/
│   ├── categorias/
│   │   ├── actualizar.php
│   │   ├── crear.php
│   │   ├── estado.php
│   │   ├── listar.php
│   │   └── obtener.php
│   ├── documentos/
│   │   ├── actualizar.php
│   │   ├── crear.php
│   │   ├── eliminar.php
│   │   ├── listar.php
│   │   ├── obtener.php
│   │   └── visualizar.php
│   ├── encuestas/
│   │   ├── actualizar.php
│   │   ├── crear.php
│   │   └── obtener.php
│   ├── funcionarios/
│   │   ├── actualizar.php
│   │   ├── crear.php
│   │   ├── estado.php
│   │   ├── listar.php
│   │   └── obtener.php
│   ├── portal/
│   │   ├── categorias.php
│   │   ├── documento.php
│   │   ├── encuesta.php
│   │   ├── encuesta resultados.php
│   │   ├── responder_encuesta.php
│   │   ├── subcategorias.php
│   │   └── visualizar.php
```

```
│   ├── seguridad/
│   │   └── csrf.php
│   ├── subcategorias/
│   │   ├── actualizar.php
│   │   ├── crear.php
│   │   ├── estado.php
│   │   ├── listar.php
│   │   └── obtener.php
│   ├── traslados/
│   │   ├── actualizar.php
│   │   ├── cancelar.php
│   │   ├── crear.php
│   │   ├── estado.php
│   │   ├── funcionarios.php
│   │   ├── listar.php
│   │   ├── obtener.php
│   │   └── vehiculos.php
│   ├── usuarios/
│   │   ├── actualizar.php
│   │   ├── crear.php
│   │   ├── estado.php
│   │   ├── listar.php
│   │   ├── login.php
│   │   ├── logout.php
│   │   ├── obtener.php
│   │   └── sesion.php
│   └── vehiculos/
│       ├── actualizar.php
│       ├── crear.php
│       ├── estado.php
│       ├── listar.php
│       ├── listar_tipos.php
│       ├── obtener.php
│       └── tipos.php
├── Backend/
│   ├── config/
│   │   ├── auth.php
│   │   ├── database.php
│   │   └── uploads.php
│   ├── database/
```

```
│   │   └── bd.sql
│   └── models/
│       ├── Categoria.php
│       ├── Documento.php
│       ├── Encuesta.php
│       ├── Funcionario.php
│       ├── Subcategoria.php
│       ├── Traslado.php
│       ├── Usuario.php
│       └── Vehiculo.php
├── docker/
│   ├── apache-sigsm.conf
│   └── uploads.ini
└── Frontend/
    └── admin/
        ├── categorias.html
        ├── categorias_crear.html
        ├── categorias_editar.html
        ├── encuesta_crear.html
        ├── encuesta_editar.html
        ├── encuesta_resultados.html
        ├── encuestas.html
        └── funcionarios.html
```

La raíz reúne el código de aplicación y la definición del entorno. Las carpetas API, Backend y Frontend contienen la funcionalidad del sistema; docker y uploads cumplen funciones de infraestructura y almacenamiento.

3. Backend: configuración, datos y reglas de negocio

3.1 Backend/config/

database.php — conexión a MySQL. Define la clase Database e implementa una instancia única mediante el patrón Singleton. El constructor privado abre una conexión PDO con juego de caracteres utf8mb4, excepciones ante errores y obtención de resultados en arreglos asociativos. getInstancia() recupera o crea la instancia y getConexion() devuelve el objeto PDO. La función independiente revertirTransaccion() intenta revertir una transacción que haya quedado abierta después de una excepción. Los parámetros de conexión provienen de variables de entorno en Docker o de valores predeterminados en desarrollo local.

auth.php — sesión, roles y CSRF. Contiene iniciarSesion(), usuarioAutenticado(), obtenerRolUsuario(), tieneRol(), esAdministrador(), exigirAutenticacion() y exigirAdministrador(). obtenerTokenCsrf() genera o recupera el token por sesión; verificarCsrf() comprueba la cabecera X-CSRF-Token para peticiones que modifican datos. Configura cookies HttpOnly, SameSite=Lax y Secure cuando la conexión usa HTTPS. Las sesiones autenticadas expiran tras 30 minutos sin peticiones. Al cargar auth.php, se invoca el control CSRF para los métodos pertinentes.

uploads.php — protección del almacenamiento. La función protegerCarpetaUploads() comprueba que exista una política .htaccess que prohíba acceder por HTTP directamente al directorio de documentos. Si falta, intenta crearla; si encuentra directivas distintas a las esperadas, rechaza la operación. El objetivo es que la entrega de PDF pase por API/documentos/visualizar.php o API/portal/visualizar.php, donde se controla quién puede ver el documento

3.2 Backend/models/

Los ocho modelos son clases PHP que encapsulan el acceso a datos de las principales entidades. Su constructor obtiene PDO de Database::getInstancia()->getConexion(). Los métodos representan operaciones sobre entidades, no peticiones HTTP: no reciben necesariamente cabeceras ni construyen la interfaz.

Usuario.php

Gestiona las cuentas de acceso, sus roles y sus estados laborales.

Método	Papel
buscarPorUser()	Buscar la cuenta por nombre de usuario; se utiliza al autenticar.
obtenerTodos() / obtenerPorId()	Consultar listados o una cuenta.
crear()	Insertar un usuario con contraseña tratada mediante password_hash().
actualizar() / cambiarEstado()	Modificar datos o estado laboral.
verificarPassword()	Comprobar una clave con password_verify().
ejecutarProtegido() / idRolAdministrador()	Proteger cambios que podrían dejar al sistema sin administrador activo.

La regla de no eliminar funcionalmente al último administrador activo se aplica con operaciones protegidas por transacción y bloqueo de filas. Mantener esta regla en el modelo evita que dependa exclusivamente del formulario o de la API que inició la modificación.

Funcionario.php

Mantiene información de funcionarios y su posibilidad de trabajar como chofer, enfermero o ambas funciones. Métodos: obtenerTodos(), buscarPorId(), buscarPorCi(), crear(), actualizar(), cambiarEstado(), validarCambioLaboral() y bloquearFuncionario().

El sistema distingue estado laboral (activo, inactivo o desvinculado) de estado operativo (libre u ocupado). Los cambios de habilitación o estado deben respetar las asignaciones a traslados activos. La misma persona se representa en una sola tabla con indicadores de habilitación; no se duplica en tablas separadas de choferes y enfermeros.

Vehiculo.php

Gestiona la flota. Métodos: obtenerTodos(), obtenerPorId(), crear(), actualizar(), cambiarEstado(), obtenerTipos() y modificarConControlDeTraslados().

La matrícula, el tipo de vehículo y sus estados se almacenan en MySQL. modificarConControlDeTraslados() protege modificaciones que entrarían en conflicto con un traslado vigente. La ocupación operativa no debería establecerse arbitrariamente desde una pantalla administrativa: la gestión del traslado es quien reserva y libera los recursos.

Traslado.php

Es el modelo con más lógica operativa. Relaciona solicitudes, vehículos, personal y el objeto transportado.

Método	Papel
obtenerTodos()	Listar traslados uniendo datos de vehículos, tipos, pacientes y otros elementos.
obtenerPorId() / existe()	Consultar un traslado y sus funcionarios o comprobar su existencia.
obtenerVehiculosDisponibles()	Obtener recursos móviles activos y libres.
obtenerFuncionariosDisponibles()	Obtener personal apto y libre.
funcionarioEnTurno()	Comprobar compatibilidad del horario con la partida.
crear()	Validar condiciones y crear una solicitud con sus relaciones, reservando recursos.
actualizar()	Modificar información cuando el estado lo permite.
avanzarEstado()	Ejecutar la siguiente transición del circuito de trazabilidad.
cancelar()	Cancelar antes de salir o abortar si ya se registró la salida.
liberarRecursos()	Método privado para dejar libres vehículo y funcionarios al concluir o cancelar.

Reglas relevantes: los traslados de pacientes requieren ambulancia; los funcionarios deben tener la habilitación adecuada, estar activos, libres y de turno; un recurso no debe asignarse simultáneamente a dos operaciones incompatibles. Para garantizarlo, crear(), avanzarEstado() y cancelar() emplean transacciones y consultas SELECT ... FOR UPDATE donde corresponde.

El circuito principal es:

```
solicitado -> aprobado -> en_curso -> llegado
          -> regreso -> retornado -> finalizado

Terminación anticipada:
   sin hora de salida -> cancelado
   con hora de salida -> abortado
```

Al finalizar, cancelar o abortar, se liberan los recursos asignados. Los campos horarios (hora_salida, hora_llegada, hora_regreso, hora_retorno, hora_finalizado) permiten reconstruir el seguimiento.

Categoria.php

Gestiona categorías del portal documental. Métodos: obtenerTodas(), obtenerPorId(), crear(), actualizar(), cambiarEstado(), existe(), desvincularSubcategorias() y validarEstado().

La relación jerárquica es categoría → subcategoría → documento/encuesta. Si se desvincula una categoría, se actualizan también sus subcategorías; esto se controla transaccionalmente para evitar un árbol de publicación incoherente.

Subcategoria.php

Administra subcategorías y verifica su relación con las categorías. Métodos: obtenerTodas(), obtenerPorId(), crear(), actualizar(), cambiarEstado() y validarEstado().

Antes de ciertas operaciones, comprueba el estado de la categoría padre. La disponibilidad en el portal depende de la situación tanto de la subcategoría como de su categoría.

Documento.php

Gestiona metadatos de documentos, no el almacenamiento de bytes de PDF dentro de MySQL. Métodos: obtenerTodos(), obtenerPorId(), crear(), actualizar(), darDeBaja() y subcategoriaDisponible().

El archivo PDF vive en uploads/documentos/; el registro de la base conserva título, ubicación y relación con la subcategoría. darDeBaja() realiza una baja lógica, por lo que el registro no desaparece físicamente. Las API de carga se ocupan de recibir y validar el archivo; el modelo se ocupa del registro SQL.

Encuesta.php

Administra encuestas, preguntas y opciones de respuesta. Métodos destacados: obtenerTodas(), obtenerPorId(), obtenerPreguntas(), obtenerOpciones(), crear(), crearCompleta(), reemplazarCompleta(), guardarPreguntas(), actualizar(), retirar(), subcategoriaDisponible() y obtenerActivaPorSubcategoria().

crearCompleta() inserta una encuesta y su estructura en una transacción. reemplazarCompleta() conserva la encuesta anterior como versión retirada y crea una nueva estructura: así no se mezclan respuestas históricas con preguntas editadas. El registro anónimo de respuestas y el cómputo de resultados se encuentran, actualmente, en endpoints del portal.

3.3 Backend/database/bd.sql

Define el esquema sigsm, sus claves primarias y foráneas y registros iniciales. Contiene 17 tablas, agrupables así:

Grupo	Tablas
Acceso y seguridad	roles, usuarios, intentos_login
Personal y movilidad	funcionarios, tipos_vehiculo, vehiculos
Trazabilidad	traslados, paciente, otro, traslado_funcionario
Documentación	categorias, subcategorias, documentos
Encuestas	encuestas, preguntas, opciones_pregunta, respuestas

Las claves foráneas vinculan entidades sin repetir todos sus campos. Por ejemplo, traslados.id_vehiculo referencia a vehiculos; traslado_funcionario relaciona una solicitud con personal y su rol; documentos referencia una subcategoría. Las consultas JOIN reconstruyen la información necesaria para mostrarla.

Importante: bd.sql es una definición y carga inicial de base de datos, no un endpoint que se ejecute en cada petición. Docker lo monta como script de inicialización de MySQL para un volumen de datos nuevo.

4. API: puntos de entrada HTTP

La carpeta API/ se organiza por recurso o módulo. Los scripts PHP reciben solicitudes GET, POST, PUT o PATCH según su implementación, verifican datos y derechos, llaman a modelos o realizan consultas y devuelven el resultado. La respuesta habitual es JSON, excepto los endpoints que transmiten PDF.

4.1 API/usuarios/

Archivo	Responsabilidad
login.php	Procesa credenciales, controla intentos fallidos y establece la sesión.
logout.php	Destruye o invalida los datos de autenticación de la sesión.
sesion.php	Informa si la sesión existe y si cumple el rol solicitado.
listar.php	Obtiene el listado administrativo de usuarios.
obtener.php	Consulta un usuario específico.
crear.php	Valida los campos recibidos y solicita Usuario::crear().
actualizar.php	Procesa las modificaciones de una cuenta.
estado.php	Cambia el estado laboral con las reglas del modelo.

login.php merece una mención aparte: utiliza la tabla intentos_login para limitar a cinco intentos fallidos por usuario en una ventana de quince minutos, con bloqueo temporal. Al autenticar, PHP almacena en $_SESSION los datos necesarios para las verificaciones posteriores. Las modificaciones de usuarios están reservadas a administradores.

4.2 API/funcionarios/

Archivo	Responsabilidad
listar.php	Lista funcionarios.
obtener.php	Obtiene información de uno.
crear.php	Valida e inserta un funcionario.
actualizar.php	Actualiza datos y habilitaciones.
estado.php	Cambia el estado laboral.

El modelo Funcionario comprueba reglas que una validación visual no puede garantizar: disponibilidad y participación en traslados activos. Estos endpoints corresponden a operaciones administrativas.

4.3 API/vehiculos/

Archivo	Responsabilidad
listar.php	Lista la flota.
obtener.php	Obtiene un vehículo.
crear.php	Da de alta un vehículo.
actualizar.php	Actualiza datos del vehículo.
estado.php	Cambia su estado laboral.
listar_tipos.php	Devuelve los tipos de vehículo.
tipos.php	Otra ruta de consulta de los tipos.

La consulta de tipos permite llenar los selectores del formulario sin escribir opciones directamente en HTML. Las reglas de modificación se apoyan en Vehiculo.php.

4.4 API/traslados/

Archivo	Responsabilidad
listar.php	Lista traslados registrados.
obtener.php	Consulta los detalles de uno.
crear.php	Valida la solicitud y llama a Traslado::crear().
actualizar.php	Modifica datos generales mientras esté permitido.
estado.php	Solicita un avance de estado mediante avanzarEstado().
cancelar.php	Ejecuta la cancelación o el aborto de un traslado.
funcionarios.php	Devuelve funcionarios libres y separa los aptos como choferes/enfermeros.
vehiculos.php	Devuelve vehículos disponibles para una nueva solicitud.

Separar actualizar.php, estado.php y cancelar.php evita que una actualización genérica permita saltar etapas o liberar recursos sin comprobar el circuito operativo. La API controla la petición; el modelo garantiza la coherencia de datos.

4.5 API/categorias/ y API/subcategorias/

Ambos directorios incluyen cinco endpoints homólogos:

Archivo	Función
listar.php	Recuperar un conjunto de registros.
obtener.php	Consultar un registro por identificador.
crear.php	Validar y crear.
actualizar.php	Editar nombre u otros datos permitidos.
estado.php	Cambiar el estado del registro.

categorias utiliza Categoria.php; subcategorias, Subcategoria.php. El control de la relación padre-hijo se hace en los modelos y en las comprobaciones de las API, no únicamente con selectores en el navegador.

4.6 API/documentos/

Archivo	Responsabilidad
listar.php	Obtener documentos; admite parámetros de filtrado.
obtener.php	Consultar metadatos de un documento.
crear.php	Recibir y validar el PDF, guardarlo y registrar sus metadatos.
actualizar.php	Editar los metadatos y, opcionalmente, reemplazar el archivo.
eliminar.php	Efectuar baja lógica, no borrado físico inmediato.
visualizar.php	Transmitir un PDF a un usuario autenticado.

crear.php y actualizar.php manejan carga de archivos con multipart/form-data. La validación incluye extensión, MIME, firma del archivo y límite de 10 MB. La respuesta al subir es JSON, pero visualizar.php sirve contenido de tipo application/pdf.

4.7 API/encuestas/

Archivo	Responsabilidad
obtener.php	Obtener el listado de encuestas para un usuario autenticado.
crear.php	Validar y crear encuesta, preguntas y opciones.
actualizar.php	Gestionar modificación, reemplazo o retiro de encuestas.

La complejidad del guardado se delega a Encuesta.php, especialmente a sus métodos transaccionales de creación y reemplazo.4.8 API/portal/

Estos endpoints abastecen la interfaz pública de pacientes. No todos exigen autenticación, pero sí aplican controles sobre visibilidad y datos.

Archivo	Responsabilidad
categorias.php	Exponer categorías públicas disponibles.
subcategorias.php	Exponer subcategorías públicas habilitadas.
documento.php	Consultar documentos visibles de la subcategoría.
visualizar.php	Servir PDFs que cumplen las reglas de publicación.
encuesta.php	Obtener la encuesta activa con sus preguntas.
responder_encuesta.php	Validar y guardar respuestas anónimas.
encuesta_resultados.php	Calcular resultados de encuestas; requiere administrador autenticado.

responder_encuesta.php escribe en la tabla respuestas, verifica la relación entre encuesta, pregunta y opción y usa transacciones. También limita a cinco envíos correctos por IP cada diez minutos mediante archivos privados temporales. Se trata de una limitación básica de abuso, no de un identificador personal de quienes responden. encuesta resultados.php realiza consultas SQL para agrupar votos y calcular porcentajes. Es una excepción concreta a la regla de que las consultas estén concentradas en models/.

4.9 API/seguridad/

csrf.php: devuelve el token CSRF de la sesión para que JavaScript lo envíe en solicitudes protegidas. Es una ruta técnica, no una pantalla de usuario.

5. Frontend: interfaz y lógica en el navegador

5.1 Páginas de entrada

Archivo	Función
Frontend/index.html	Página inicial y punto de acceso visual a los sectores.
Frontend/login.html	Formulario de credenciales y comunicación con usuarios/login.php.
Frontend/portal.html	Portal público de consulta documental y encuestas.

El portal utiliza Frontend/js/portal.js, donde se organiza la interacción con las API públicas, la carga de categorías y subcategorías, los documentos y las encuestas. portal.html aporta la estructura visual que utiliza ese JavaScript.

5.2 Frontend/admin/

Contiene pantallas reservadas a funciones de administración. Las páginas de lista consultan endpoints y construyen tablas; las de creación reciben datos; las de edición consultan el registro actual antes de guardar cambios.

Archivo	Función visible / lógica relevante
index.html	Página principal del área de administración.
usuarios.html	Tabla de usuarios; cargarUsuarios() y cambiarEstado().
usuarios_crear.html	Formulario de alta y envío a API/usuarios/crear.php.
usuarios_editar.html	Consulta y modifica una cuenta; cargarUsuario().
funcionarios.html	Listado; cargarFuncionarios() y cambiarEstado().
funcionarios_crear.html	Formulario de alta, rol funcional y horario.
funcionarios_editar.html	Edición; determinarTipo() y cargarFuncionario().
vehiculos.html	Listado; cargarVehiculos() y cambiarEstado().
vehiculos_crear.html	Formulario de alta; cargarTipos() consulta las opciones disponibles.
vehiculos_editar.html	Edición; cargarTipos() y cargarVehiculo().

Archivo	Función visible / lógica relevante
categorias.html	Listado de categorías; cargarCategorias() y cambiarEstado().
categorias_crear.html	Formulario para registrar categorías.
categorias_editar.html	Edición y gestión de subcategorías; cargarDatosCategoria().
subcategorias_crear.html	Alta de subcategoría; cargarCategorias() para elegir su padre.
subcategorias_editar.html	Edición; cargarDatos() y actualizarBoton().
encuestas.html	Listado de encuestas; cargarEncuestas() y listarEncuestas().
encuesta_crear.html	Constructor dinámico; crearPregunta(), agregarOpcion(), guardarEncuesta().
encuesta_editar.html	Carga, modificación y retirada; cargarEncuesta().
encuesta_resultados.html	Consulta y representa resultados; cargarResultados().

La lógica específica de muchas páginas se encuentra en bloques <script> dentro del propio HTML. Esto significa que no toda la lógica JavaScript está centralizada en Frontend/js/: esa carpeta contiene principalmente funciones reutilizables y de infraestructura visual.

5.3 Frontend/traslados/

Archivo	Función visible / funciones principales
index.html	Lista y sigue traslados; cargarTraslados(), mostrarTraslados(), siguienteEstado(), avanzarTraslado(), cancelarTraslado().
nuevopaciente.html	Formulario de traslado de paciente; cargarRecursos() y enTurno().
nuevootro.html	Formulario de traslado de otros elementos; cargarRecursos() y enTurno().

La comprobación de horarios hecha en el Frontend ayuda a la experiencia de uso, pero no sustituye la comprobación del servidor. La disponibilidad puede cambiar desde otra sesión y debe volver a validarse en Traslado::crear().

5.4 Frontend/documentos/

Archivo	Función visible / funciones principales
index.html	Listado, filtros, visor y baja lógica; cargarDocumentos(), mostrarDocumentos(), visualizarDocumento() y darDeBajaDocumento().
documento_subir.html	Recoge título, subcategoría y PDF; cargarDatos().
documento_editar.html	Recupera los metadatos existentes y permite editarlos; cargarDatos() y cargarSubcategorias().

Los formularios de PDF no envían el archivo como JSON: utilizan los mecanismos de carga de archivos del navegador, como FormData.

5.5 Frontend/dashboard/

index.html: tablero de resumen. Usa cargarTraslados(), cargarDocumentos(), cargarDashboard(), mostrarFechaHoy() y mostrarSaludo() para mostrar indicadores como traslados pendientes, en proceso, finalizados durante el día, prioridad alta y estados de publicación documental. Los contadores se calculan en JavaScript sobre datos obtenidos de las API, no en una tabla exclusiva de indicadores.

5.6 Frontend/js/

Archivo	Funciones o responsabilidad
auth.js	verificarSesion(), configurarLogout(), cerrarSesion(); navegación para usuarios autenticados.
admin-auth.js	verificarAdministrador(), cerrarSesion(); restricciones visuales del administrador.
csrf.js	Intercepta fetch() del mismo origen en POST/PUT/PATCH/DELETE; obtiene token y envía X-CSRF-Token.
admin-sidebar.js	Construye el menú lateral del administrador y resalta la sección activa.
operator-sidebar.js	Construye el menú del entorno operativo.
header.js	Construye el encabezado compartido y controles visuales de tema/idioma.

Archivo	Funciones o responsabilidad
preferencias.js	obtenerPreferencia(), aplicarTema(), aplicarIdioma(), traducirPagina(); conserva preferencias en el navegador.
portal-idioma.js	traducirNodo(), traducirAtributos(), traducirInterfaz(); traduce textos de interfaz del portal sin alterar los datos recuperados.
portal.js	Implementa la interacción del portal del paciente con las API públicas, los documentos, el visor PDF y las encuestas.

Las comprobaciones de sesión en JavaScript sirven para redirigir y mostrar una interfaz coherente, pero no son una medida suficiente de autorización. El servidor debe verificar de nuevo cada acción sensible.

5.7 Frontend/css/ y Frontend/img/

Archivo	Función
css/layout.css	Distribución, estructura de página y navegación.
css/components.css	Estilos reutilizables de controles, tablas, formularios y botones.
css/tema.css	Variantes y adaptación al tema visual.
css/portal.css	Estilos específicos del portal público.
img/Logo_Hc.png	Identidad gráfica del hospital.
img/free-informacion-icon-svg-download-png-1604565.png	Recurso gráfico informativo.

Los CSS y las imágenes son recursos estáticos: no acceden a MySQL ni procesan solicitudes de negocio.

6. Infraestructura, ejecución y almacenamiento

Archivo	Responsabilidad
Dockerfile	Construye la imagen sobre php:8.3-apache-bookworm, instala extensiones PHP, configura Apache y copia la aplicación.
compose.yml	Define los servicios web y mysql, las variables de entorno, el estado de salud y los volúmenes.
.env.example	Plantilla de configuración; no debe usarse para publicar credenciales reales.
.dockerignore	Evita incorporar al contexto de construcción archivos prescindibles o sensibles.
docker/apache-sigsm.conf	Configura Apache en Docker: deshabilita índices, permite .htaccess y bloquea rutas internas y archivos sensibles.
docker/uploads.ini	Configura los límites de PHP para las cargas: upload_max_filesize=10M y post_max_size=12M.
README.md	Guía de instalación local con XAMPP, separada de la configuración Docker.
uploads/.htaccess	Niega acceso web directo a PDF y subdirectorios de carga.
uploads/documentos/	Directorio destinado a los PDF que se almacenan durante la ejecución.
INSTRUCCIONES_DOCKER_VM.md	Guía operativa de despliegue de la aplicación en una máquina virtual con Docker.

compose.yml configura dos contenedores: web (Apache + PHP) y mysql (MySQL 8.4). web accede a la base con el nombre interno de servicio mysql, no a través de localhost del contenedor. Las credenciales de MySQL se suministran mediante variables de entorno; el puerto externo del servidor web es configurable, con valor predeterminado 8080.

Dos volúmenes dan persistencia a los datos: sigsm_mysql_data para la base y sigsm_uploads para los archivos.

Sin estos volúmenes, recrear los contenedores podría provocar pérdida del estado persistente. El archivo bd.sql se usa para inicializar una instalación de MySQL cuando su directorio de datos aún no existe; no es un mecanismo automático de migración para bases ya inicializadas.

Protección de PDF: uploads/.htaccess deniega el acceso HTTP directo a la carpeta de archivos subidos cuando Apache permite aplicar esas reglas. Las API autorizadas leen el archivo desde el sistema de archivos y lo transmiten después de verificar las condiciones aplicables. En Docker, docker/apache-sigsm.conf habilita AllowOverride All y añade restricciones para rutas internas y archivos sensibles; estas restricciones adicionales no se trasladan automáticamente a XAMPP. En XAMPP debe verificarse la configuración de Apache y comprobar el bloqueo HTTP con un PDF existente. Los límites de carga del archivo docker/uploads.ini tampoco se aplican automáticamente a XAMPP.